

José Antonio Pastor Rivera – RA5b 'Del código al diagrama'

Contexto inicial

Acabas de incorporarte a un proyecto existente y el equipo de desarrollo anterior no dejó documentación. Tienes acceso al código fuente, pero necesitas entender cómo interactúan las clases entre sí. Tu tarea es analizar el código Java proporcionado y generar el diagrama UML que lo representa fielmente.

Primera parte: Análisis previo del Código Fuente

Composición: ¿Dónde se crean los objetos?

Análisis: La creación del objeto *Motor* se realiza internamente dentro del constructor de la clase *Coche*, lo que establece una relación de composición (dependencia de ciclo de vida).

Justificación en código: Archivo *Coche.java*, Línea 6:

this.motor = new Motor(caballos); // Composición fuerte

Herencia: ¿Qué clases extienden de otras?

Análisis: La clase concreta *Coche* hereda de la clase abstracta *Vehiculo*, adoptando sus atributos (*matricula*) e implementando sus métodos abstractos (*arrancar*).

Justificación en código: Archivo *Coche.java*, Línea 1:

public class Coche extends Vehiculo {

(Además, se llama al constructor del padre en la Línea 5: *super(matricula);*)

Dependencia: ¿Qué métodos reciben objetos como parámetros?

Análisis: La clase *Mecanico* tiene una dependencia de uso con la clase *Coche*, ya que no lo guarda como atributo propio, sino que lo recibe temporalmente a través de un método para interactuar con él.

Justificación en código: Archivo *Mecanico.java*, Línea 8:

public void reparar(Coche c) {

Acceso: ¿Qué atributos tienen métodos get y set?

Análisis: Se aplica un encapsulamiento estricto. Todos los atributos están protegidos/privados y se accede a ellos así:

- En *Vehiculo.java*: El atributo *matricula* tiene *getMatricula()* (Línea 10) y *setMatricula()* (Línea 11).
- En *Motor.java*: El atributo *caballos* tiene *getCaballos()* (Línea 13) y *setCaballos()* (Línea 14). El atributo booleano *encendido* tiene su getter *isEncendido()* (Línea 15).
- En *Mecanico.java*: El atributo *nombre* tiene *getNombre()* (Línea 13) y *setNombre()* (Línea 14).

Nota clave de diseño: En *Coche.java*, el atributo *motor* tiene getter (*getMotor()* en la Línea 14),

pero no tiene setter. Esto es completamente intencionado y justifica la Composición: como el motor nace y muere con el coche, no se debe permitir cambiar el motor desde fuera una vez construido el coche.

Tercera Parte: Razonamiento

¿Por qué la relación entre Coche y Motor es una composición y no una agregación? Indícalo basándote en la línea de código exacta del constructor.

Es una relación de composición porque el ciclo de vida del objeto *Motor* depende estrictamente del objeto *Coche*. El *motor* no se crea fuera y se le pasa al *coche*, sino que el coche "nace" con su propio *motor*. Esto se demuestra en el constructor de la clase *Coche.java*, donde se instancia el objeto internamente utilizando la palabra reservada *new*. La línea de código exacta es:

```
this.motor = new Motor(caballos); // Composición fuerte
```

¿Qué tipo de flecha has usado para la relación entre Mecanico y Coche, y por qué?

Flecha discontinua (..> Dependencia): Significa "A usa a B". Se usa cuando B solo aparece como parámetro de un método, o se crea temporalmente dentro de una función, pero no es un atributo.

¿Qué implica que la clase Vehiculo sea abstracta?

Implica que no se pueden crear instancias directas de esta clase (es decir, el compilador daría un error si intentáramos hacer *new Vehiculo("1234ABC")*). Actúa exclusivamente como una superclase o "plantilla" base para obligar a sus clases hijas (como *Coche*) a heredar la estructura común (como el atributo *matricula*) y a implementar obligatoriamente el comportamiento de los métodos abstractos, en este caso el método:

```
public abstract void arrancar();
```

CONCLUSIÓN FINAL

A través de este análisis queda patente que el propio código fuente es la mejor fuente de verdad. Al identificar correctamente las dependencias de ciclo de vida, la herencia y las relaciones de uso temporal, hemos logrado transformar un proyecto opaco en un modelo UML claro y documentado, sentando las bases para su futuro desarrollo y mantenimiento.